

Практики 13–14: Систематизация методов многомерного анализа и внедрение в production

СамГТУ, кафедра Высшей математики
Преподаватель: М.В. Долгополов

2026

Содержание

1 Введение	3
2 Практика 13: Сравнительный анализ и выбор оптимального метода	3
2.1 Диагностика данных и автовыбор метода	3
2.1.1 Критерии диагностики	3
2.1.2 Формула КМО	3
2.1.3 Реализация на Python	4
2.2 Сравнительная таблица методов	4
2.3 Задания для самостоятельной работы (с решениями)	5
2.3.1 Задание 1: Customer Behavior (числовые + категориальные) . . .	5
2.3.2 Задание 2: Swiss Roll – сравнение PCA, Isomap, KernelPCA	5
2.3.3 Задание 3: Wine Quality – FA vs PCA	6
3 Практика 14: Production Pipeline и внедрение	6
3.1 Универсальный класс снижения размерности	6
3.2 Case Study: Retail Analytics (сегментация клиентов)	7
3.2.1 Данные	7
3.2.2 Пайплайн	7
3.2.3 Интерпретация кластеров	8
3.3 Развертывание (Flask API + Streamlit)	8
4 Тренды 2026+ и оценка ROI	9
4.1 Топ-5 трендов	9
4.2 ROI по отраслям	9
A Приложение 1: Правильные ответы на онлайн-тест с обоснованиями	10
B Приложение 2: Тезаурус понятий и сокращений (практики 13–14)	10
C Приложение 3: Мастерское подробное решение одного из заданий (Retail Analytics)	11
C.1 Постановка задачи	11
C.2 Полный код с комментариями	11
C.3 Анализ профилей	12

С.4	Маркетинговые рекомендации	13
С.5	Заключение по заданию	13

1 Введение

Практики 13–14 завершают курс «Анализ данных» и посвящены систематизации всех изученных методов снижения размерности (PCA, FA, CA, MCA, MDS, Kernel PCA, Isomap), выработке навыка обоснованного выбора метода в зависимости от характеристик данных, а также созданию автоматизированных пайплайнов для внедрения в промышленную среду (production).

Рассматриваются:

- Диагностика данных (тест линейности, КМО, соотношение числовых/категориальных признаков).
- Сравнительная таблица методов по 7 критериям.
- Универсальный класс снижения размерности с автовыбором метода.
- Case Study: сегментация клиентов (Retail Analytics) на основе PCA + MCA + K-means.
- Развертывание решения через Flask API и Streamlit.
- Актуальные тренды 2026+ и оценка ROI для различных отраслей.

2 Практика 13: Сравнительный анализ и выбор оптимального метода

2.1 Диагностика данных и автовыбор метода

Цель: Написать функцию, которая по структуре входных данных рекомендует наиболее подходящий метод многомерного анализа.

2.1.1 Критерии диагностики

1. Тип данных (числовые / категориальные / частоты).
2. Пригодность для факторного анализа: мера КМО (Kaiser-Meyer-Olkin) — чем ближе к 1, тем лучше; обычно порог >0.7 .
3. Линейность связи между переменными — можно оценить по ранговой корреляции расстояний.

2.1.2 Формула КМО

$$KMO = \frac{\sum \sum_{i \neq j} r_{ij}^2}{\sum \sum_{i \neq j} r_{ij}^2 + \sum \sum_{i \neq j} p_{ij}^2},$$

где r_{ij} — парные коэффициенты корреляции Пирсона, p_{ij} — частные корреляции.

2.1.3 Реализация на Python

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.decomposition import PCA, FactorAnalysis
4 from sklearn.manifold import Isomap
5 import prince
6
7 def diagnose_data(df):
8     """
9
10
11     """
12     numeric_cols = df.select_dtypes(include=[np.number]).columns
13     cat_cols = df.select_dtypes(include=['object', 'category']).columns
14
15     # :
16     # -> MCA
17     if len(cat_cols) > len(numeric_cols):
18         return "MCA (Multiple Correspondence Analysis)"
19
20     # ,
21     # KMO (
22     if len(numeric_cols) > 1:
23         # KMO (
24         # factor_analyzer)
25         kmo = np.abs(df[numeric_cols].corr().values).mean()
26         if kmo > 0.7:
27             return "FA (Factor Analysis)"
28
29     # ( :
30     # )
31     return "PCA (Principal Component Analysis)"

```

2.2 Сравнительная таблица методов

Критерий	PCA	FA	CA	MCA	MDS	KPCA	Isomap
Тип данных	Числовые	Числовые	Частоты	Категории	Расстояния	Числовые	Любые
Линейность					Любая		
Латентные факторы							
Масштабируемость							
Интерпретируемость							
Сложность (big-O)	$O(p^3)$	$O(p^3 + iter)$	$O(\min(n, p)^3)$	$O((np)^3)$	$O(n^3)$	$O(n^2p)$	$O(n^2k)$
Библиотека	sklearn	sklearn	prince	prince	sklearn	sklearn	sklearn

Таблица 1: Сравнительная таблица методов многомерного анализа

2.3 Задания для самостоятельной работы (с решениями)

2.3.1 Задание 1: Customer Behavior (числовые + категориальные)

Исходные данные: 10 числовых признаков (расходы, доход, время) и 5 категориальных (регион, тип продукта, канал продаж). Требуется визуализировать кластеры покупателей.

Решение:

```
1 from sklearn.decomposition import PCA
2 from sklearn.preprocessing import StandardScaler
3 import prince
4 from sklearn.cluster import KMeans
5
6 # -> PCA
7 scaler = StandardScaler()
8 X_num_scaled = scaler.fit_transform(df[numeric_cols])
9 pca = PCA(n_components=2)
10 X_pca = pca.fit_transform(X_num_scaled)
11
12 # -> MCA
13 mca = prince.MCA(n_components=2)
14 X_mca = mca.fit_transform(df[cat_cols]).values
15
16 #
17 X_combined = np.hstack([X_pca, X_mca])
18 kmeans = KMeans(n_clusters=4, random_state=42)
19 clusters = kmeans.fit_predict(X_combined)
20
21 #
22 import matplotlib.pyplot as plt
23 plt.scatter(X_pca[:,0], X_pca[:,1], c=clusters, cmap='viridis')
24 plt.title("PCA = ")
25 plt.show()
```

2.3.2 Задание 2: Swiss Roll – сравнение PCA, Isomap, KernelPCA

Решение:

```
1 from sklearn.datasets import make_swiss_roll
2 X, _ = make_swiss_roll(n_samples=1000, noise=0.1)
3
4 pca = PCA(n_components=2).fit_transform(X) #
5 iso = Isomap(n_neighbors=10, n_components=2).fit_transform(X) #
6 kpca = KernelPCA(kernel='rbf', gamma=0.1, n_components=2).
    fit_transform(X)
7
8 # 2D
9 fig, axes = plt.subplots(1,3, figsize=(15,4))
10 axes[0].scatter(pca[:,0], pca[:,1]); axes[0].set_title("PCA")
11 axes[1].scatter(iso[:,0], iso[:,1]); axes[1].set_title("Isomap")
```

```

12 axes[2].scatter(kpca[:,0], kpca[:,1]); axes[2].set_title("Kernel PCA")
    )

```

Вывод: PCA плохо разворачивает многообразие; Isomap и Kernel PCA справляются лучше, причём Isomap даёт наиболее равномерное покрытие.

2.3.3 Задание 3: Wine Quality – FA vs PCA

Решение:

```

1 from sklearn.decomposition import FactorAnalysis
2 fa = FactorAnalysis(n_components=3, rotation='varimax')
3 fa.fit(X_scaled)
4 loadings = fa.components_.T    #
5
6 #                               :                               1
7                               ,                               ,                               2
8                               ,                               3                               .

```

3 Практика 14: Production Pipeline и внедрение

3.1 Универсальный класс снижения размерности

```

1 from sklearn.base import BaseEstimator, TransformerMixin
2 from sklearn.pipeline import Pipeline
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.decomposition import PCA, FactorAnalysis, KernelPCA
5 from sklearn.manifold import Isomap
6 import prince
7
8 class UniversalDimReduction(BaseEstimator, TransformerMixin):
9     def __init__(self, method='auto', n_components=2):
10         self.method = method
11         self.n_components = n_components
12         self.model = None
13
14     def fit(self, X, y=None):
15         if self.method == 'auto':
16             self.method = self._choose_method(X)
17         if self.method == 'pca':
18             self.model = Pipeline([('scaler', StandardScaler()),
19                                   ('pca', PCA(n_components=self.
20                                               n_components))])
21         elif self.method == 'fa':
22             self.model = Pipeline([('scaler', StandardScaler()),
23                                   ('fa', FactorAnalysis(n_components
24                                                         =self.n_components))])
25         elif self.method == 'mca':
26             self.model = prince.MCA(n_components=self.n_components)
27         elif self.method == 'isomap':
28             self.model = Isomap(n_components=self.n_components)
29         self.model.fit(X)

```

```

28         return self
29
30     def transform(self, X):
31         return self.model.transform(X)
32
33     def _choose_method(self, X):
34         #
35         n_cat = X.select_dtypes(include=['object', 'category']).shape
36             [1]
37         n_num = X.select_dtypes(include=[np.number]).shape[1]
38         if n_cat > n_num:
39             return 'mca'
40         else:
41             return 'pca' # KMO

```

3.2 Case Study: Retail Analytics (сегментация клиентов)

3.2.1 Данные

Таблица транзакций (100 тыс. строк). Числовые признаки: сумма покупки, количество товаров, время в магазине. Категориальные: категория товара, день недели, метод оплаты.

3.2.2 Пайплайн

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.cluster import KMeans
4 from sklearn.preprocessing import StandardScaler
5 import prince
6 from sklearn.decomposition import PCA
7
8 df = pd.read_csv('retail.csv')
9 num_cols = ['total_spend', 'num_items', 'time_in_store']
10 cat_cols = ['category', 'day_of_week', 'payment_method']
11
12 # -> PCA
13 scaler = StandardScaler()
14 X_num = scaler.fit_transform(df[num_cols])
15 pca = PCA(n_components=3)
16 X_pca = pca.fit_transform(X_num)
17
18 # -> MCA
19 mca = prince.MCA(n_components=3)
20 X_mca = mca.fit_transform(df[cat_cols]).values
21
22 #
23 X_combined = np.hstack([X_pca, X_mca])
24
25 #
26 kmeans = KMeans(n_clusters=5, random_state=42)
27 clusters = kmeans.fit_predict(X_combined)

```

```

28 df['cluster'] = clusters
29
30 #
31 profile = df.groupby('cluster').agg({
32     'total_spend': 'mean',
33     'num_items': 'mean',
34     'category': lambda x: x.mode()[0],
35     'payment_method': lambda x: x.mode()[0]
36 })
37 print(profile)

```

3.2.3 Интерпретация кластеров

Кластер	Профиль	Стратегия маркетинга
0	Молодые, карты, вечер	Email-промо, ночные скидки
1	Пожилые, наличные, утро	Купоны в почтовые ящики
2	Семьи, большие покупки	Скидки на опт
3	Студенты, малые покупки	Программа лояльности
4	Премиум-сегмент	VIP-карты, персональные предложения

Таблица 2: Профили кластеров клиентов

3.3 Развертывание (Flask API + Streamlit)

```

1 # API (Flask)
2 from flask import Flask, request, jsonify
3 app = Flask(__name__)
4
5 @app.route('/predict', methods=['POST'])
6 def predict():
7     data = request.json
8     X = pd.DataFrame(data)
9     reducer = UniversalDimReduction(method='auto')
10    reducer.fit(X)
11    X_red = reducer.transform(X)
12    kmeans = KMeans(n_clusters=5).fit(X_red)
13    clusters = kmeans.predict(X_red)
14    return jsonify({'clusters': clusters.tolist()})
15
16 # Streamlit frontend
17 import streamlit as st
18 st.title("Классификация покупок клиентов")
19 uploaded = st.file_uploader("Загрузите файл CSV")
20 if uploaded:
21     df = pd.read_csv(uploaded)
22     method = st.selectbox("Выбор метода", ['auto', 'pca', 'fa', 'mca', 'isomap'])
23     reducer = UniversalDimReduction(method=method)
24     reducer.fit(df)
25     st.write(f"Результат: {reducer.method}")
26     st.scatter_chart(reducer.transform(df))

```


4 Тренды 2026+ и оценка ROI

4.1 Топ-5 трендов

1. **AutoML** для снижения размерности – интеграция с H2O, DataRobot.
2. **Real-time streaming PCA** – обработка потоков через Apache Kafka + sklearn.
3. **Federated Factor Analysis** – распределённый FA без передачи сырых данных.
4. **Explainable MDS** – интерпретация расстояний с использованием SHAP.
5. **Quantum PCA** – квантовые алгоритмы для размерности $>10k$.

4.2 ROI по отраслям

Сфера	Метод	Экономия / Прирост
Retail	MCA	+15% конверсия
Healthcare	FA	-20% ошибок диагностики
Manufacturing	PCA	-30% брака
Finance	KPCA	+10% точность прогноза

Таблица 3: Ожидаемый ROI при внедрении аналитических решений

А Приложение 1: Правильные ответы на онлайн-тест с обоснованиями

1. **Вопрос:** Какой метод автоматически выберет `diagnose_data()` для набора данных с 8 числовыми признаками и высоким КМО (>0.7)?

Ответ: FA (Factor Analysis).

Обоснование: Высокое значение КМО свидетельствует о том, что данные пригодны для факторного анализа, а наличие числовых признаков без преобладания категориальных делает FA предпочтительнее PCA, так как FA выявляет латентные структуры.

2. **Вопрос:** Какой метод является оптимальным для развёртывания нелинейного многообразия (Swiss roll)?

Ответ: Isomap.

Обоснование: Isomap восстанавливает геодезические расстояния через граф ближайших соседей и применяет MDS, что идеально для данных типа «швейцарский руль».

3. **Вопрос:** Для сегментации клиентов на основе категориальных анкетных данных лучше всего подходит:

Ответ: MCA (Multiple Correspondence Analysis).

Обоснование: MCA является обобщением CA на случай нескольких категориальных переменных и напрямую работает с dummy-кодированными данными, выделяя ассоциации между категориями.

4. **Вопрос:** Какой метод позволяет выделить интерпретируемые латентные факторы (например, "метаболический синдром")?

Ответ: FA (Factor Analysis).

Обоснование: Факторный анализ моделирует наблюдаемые переменные как линейные комбинации скрытых факторов, что позволяет содержательно назвать факторы после вращения (Varimax).

5. **Вопрос:** Для визуализации матрицы попарных расстояний между 50 городами следует применить:

Ответ: MDS (Multidimensional Scaling).

Обоснование: MDS принимает на вход матрицу расстояний и строит конфигурацию точек в низкоразмерном пространстве, сохраняя исходные расстояния с минимальной ошибкой.

В Приложение 2: Тезаурус понятий и сокращений (практики 13–14)

Термин аббр.	/	Определение
PCA		Principal Component Analysis – линейное преобразование, максимизирующее дисперсию проекций.
FA		Factor Analysis – статистическая модель, объясняющая корреляции через латентные факторы.

CA	Correspondence Analysis – анализ контингентных таблиц на основе χ^2 -расстояния.
MCA	Multiple Correspondence Analysis – обобщение CA на случай нескольких категориальных переменных (анкеты).
MDS	Multidimensional Scaling – построение конфигурации по матрице попарных расстояний.
Isomap	Изометрическое отображение – нелинейное сжатие через граф k-ближайших соседей и геодезические расстояния.
Kernel PCA	Ядерный PCA – нелинейное обобщение PCA через отображение в пространство признаков.
KMO	Kaiser-Meyer-Olkin – мера адекватности выборки для факторного анализа; > 0.7 – хорошо.
EVR	Explained Variance Ratio – доля объяснённой дисперсии для главных компонент.
ROI	Return on Investment – экономический эффект от внедрения аналитического решения.
Production pipeline	Автоматизированный конвейер обработки данных, включающий предобработку, снижение размерности, кластеризацию и API.

С Приложение 3: Мастерское подробное решение одного из заданий (Retail Analytics)

С.1 Постановка задачи

Дана таблица транзакций супермаркета (100 тыс. строк). Признаки:

- Числовые: `total_spend` (сумма покупки, руб), `num_items` (количество товаров), `time_in_store` (время в магазине, мин).
- Категориальные: `category` (тип товара: «продукты», «бытовая химия», «электроника»), `day_of_week` (пн–вс), `payment_method` (наличные, карта, онлайн).

Требуется сегментировать клиентов на 5 кластеров и предложить маркетинговые стратегии.

С.2 Полный код с комментариями

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.decomposition import PCA
6 from sklearn.cluster import KMeans
7 import prince
8
9 # 1.
10 df = pd.read_csv('retail.csv')
```

```

11 num_cols = ['total_spend', 'num_items', 'time_in_store']
12 cat_cols = ['category', 'day_of_week', 'payment_method']
13
14 # 2.
15 scaler = StandardScaler()
16 X_num = scaler.fit_transform(df[num_cols]) #
17         (100000, 3)
18 pca = PCA(n_components=3) #
19         3
20 X_pca = pca.fit_transform(X_num) #
21         (100000, 3)
22
23 # 3. MCA (one-hot)
24         hot )
25 mca = prince.MCA(n_components=3) #
26         = 3
27 X_mca = mca.fit_transform(df[cat_cols]).values #
28         (100000, 3)
29
30 # 4. PCA MCA
31 X_combined = np.hstack([X_pca, X_mca]) #
32         (100000, 6)
33
34 # 5. K-means (k=5)
35 kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
36 clusters = kmeans.fit_predict(X_combined)
37 df['cluster'] = clusters
38
39 # 6.
40 profile = df.groupby('cluster').agg({
41     'total_spend': 'mean',
42     'num_items': 'mean',
43     'category': lambda x: x.mode()[0],
44     'payment_method': lambda x: x.mode()[0]
45 })
46 print(profile)
47
48 # 7. PCA
49
50 plt.figure(figsize=(10,8))
51 plt.scatter(X_pca[:,0], X_pca[:,1], c=clusters, cmap='tab10', alpha
52             =0.5)
53 plt.colorbar(label='Cluster')
54 plt.xlabel('PC1')
55 plt.ylabel('PC2')
56 plt.title('
57
58             (
59             )')
60 plt.show()

```

С.3 Анализ профилей

Результат работы кода (пример):

cluster	total_spend	num_items	category	payment_method
0	1250.34	3.2	продукты	карта
1	340.12	1.5	электроника	наличные
2	890.45	2.7	бхтхимия	карта
3	2100.78	5.1	продукты	карта
4	78.50	1.0	электроника	наличные

С.4 Маркетинговые рекомендации

- **Кластер 0** (средний чек 1250 руб, продукты, карта) – предлагать персонализированные скидки на популярные продукты.
- **Кластер 1** (низкий чек, электроника, наличные) – стимулировать переход на безналичную оплату и увеличивать средний чек через кросс-продажи.
- **Кластер 2** (средний чек, бытовая химия) – программы лояльности на регулярные покупки.
- **Кластер 3** (высокий чек, продукты, карта) – премиальные предложения, дегустации.
- **Кластер 4** (минимальный чек, электроника, наличные) – акции на недорогие товары и обучение пользованию смартфонами для оплаты.

С.5 Заключение по заданию

Используя комбинацию РСА (для числовых) и МСА (для категориальных), удалось построить интерпретируемое шестимерное пространство, в котором кластеризация выделила экономически значимые сегменты. Полученные профили кластеров напрямую легли в основу маркетинговых стратегий, что демонстрирует практическую ценность методов многомерного анализа.